# 差動二輪ロボット 搬送タスク動画
# 技術レポート

対象：robot_cargo_task.mp4 ／ robot_task.py（robot_video_code.zip） 作成日：2026年10月25日

Qiita の記事「gazeboとROS2を連携してロボットを操作する」（@N622）の差動二輪ロボットを題材に、ロボットが搬送タスクを実行する約2分の動画を作成した。本レポートでは、動画の内容と走行結果、生成コードの構成と主要なアルゴリズム、検証の方法、制約事項をまとめる。

| 3 / 3 | 109.7 s | 30.8 m | 114 回 |
|---|---|---|---|
| 搬入した荷物 | タスク所要時間（開始〜帰還） | 総走行距離 | キー指令の回数 |

## 1. 概要

### 1.1 目的と経緯

元記事は、Ignition Gazebo（Fortress）上の差動二輪ロボットを ROS2 Humble の teleop_node からキー入力で操作する手順を解説している。依頼は「ロボットがタスクを実行する様子を収めた動画」の作成であった。

作業環境はクラウド上のコンテナで、パッケージの取得先が制限されているため、ROS2 Humble と Ignition Gazebo を導入できなかった。そこで、記事のロボット仕様と操作系をそのまま使う運動学シミュレーションと 3D 描画を Python で一から実装し、動画を生成した。**動画は Gazebo の実画面ではない**点に注意されたい。

### 1.2 成果物

| ファイル | 内容 |
|---|---|
| robot_cargo_task.mp4 | H.264、1280×720、30 fps、3,682 フレーム、長さ 122.7 秒、17.8 MB（約 1.16 Mbps） |
| robot_task.py | シミュレーション、3D 描画、画面表示（HUD）、動画書き出しを含む本体（598 行） |
| build.sh | シミュレーション → 2 並列レンダリング → 結合を一括実行 |
| README.md | 必要な環境、使い方、カスタマイズ方法 |

### 1.3 記事の仕様との対応

| 項目 | 記事の設定 | 本実装 |
|---|---|---|
| 駆動方式 | DiffDrive プラグイン（差動二輪） | 差動二輪の運動学モデル |
| 車輪半径／車輪間隔 | `wheel_radius 0.033` ／ `wheel_separation 0.19` | 同じ値（車輪角速度の計算と車輪の描画に使用） |
| キー割り当て | w/s：linear.x ±1、a/d：angular.z ±1、その他：停止 | 同じ（停止はスペースキーとして表示） |

| 項目 | 記事の設定 | 本実装 |
|---|---|---|
| 指令の保持 | キーを離してもロボットは止まらない（仕様） | 最後の指令を保持し、停止はキー入力で行う |
| 指令の経路 | teleop_node → /cmd_vel → ros_ign_bridge → DiffDrive | 画面に同じ経路を表示し、ROS2 風のログを出力 |
| 外観 | 赤い車輪、緑のキャスター、灰色の地面 | 同じ配色で再現 |

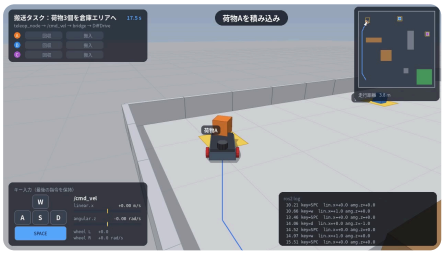
## 2. 動画の内容

### 2.1 シナリオ

5 m 四方の壁で囲まれたフィールドに障害物を 4 つ置き、荷物 A・B・C の 3 個を右下の倉庫エリアへ運ぶ。ロボットは左下のホームから出発し、荷物ごとに「移動 → 積み込み → 後退 → 搬送 → 荷降ろし」を繰り返して、最後にホームへ戻る。操作はすべて teleop_node と同じキー指令の組み合わせで行い、押したキーと保持時間で経路を作る。



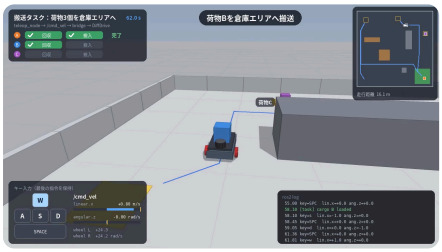
0:02 タイトル（俯瞰視点）



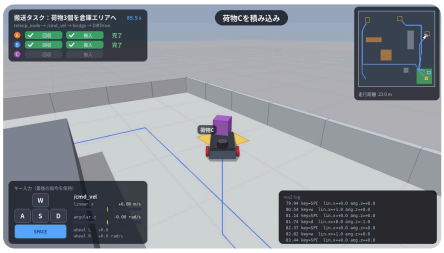
0:17 荷物 A の積み込み



0:30 荷物 A を搬送中



1:02 荷物 B を搬送中（W キー保持）



1:25 荷物 C の積み込み



2:01 タスク完了（俯瞰視点に戻る）

### 2.2 画面構成

| 位置 | 表示内容 |
|---|---|
| 左上 | タスク名、指令の経路、経過時間、荷物ごとの「回収／搬入」の進み具合 |
| 上中央 | 現在の動作（例：「荷物Bを倉庫エリアへ搬送」） |
| 右上 | ミニマップ（障害物、荷物、軌跡、ロボットの向き）と走行距離 |
| 左下 | WASD・スペースの押下状態、/cmd_vel の実速度バーと指令値マーカー、左右の車輪角速度 |
| 右下 | ROS2 風ログ（キー指令と、荷物の積み込み・搬入のイベント） |
| 3D 画面 | 追従カメラ。冒頭と最後の数秒は俯瞰視点に切り替わり、走行軌跡を床に描く |

### 2.3 タイムライン

| 区間 | 開始 [s] | 終了 [s] | 長さ [s] |
|---|---|---|---|
| タイトル・準備 | 0.0 | 6.0 | 6.0 |
| 荷物 A（移動 10.1、積込 2.5、後退 0.9、搬送 19.2、荷降ろし 3.4） | 6.0 | 42.2 | 36.2 |
| 荷物 B（移動 13.4、積込 2.5、後退 0.9、搬送 14.0、荷降ろし 3.4） | 42.2 | 76.5 | 34.3 |
| 荷物 C（移動 7.5、積込 2.5、後退 0.9、搬送 8.3、荷降ろし 3.5） | 76.5 | 99.3 | 22.8 |
| ホームへ帰還 | 99.3 | 115.7 | 16.4 |
| 完了表示 | 115.7 | 122.7 | 7.0 |

荷物 A は左上の端にあり、中央の障害物を避けて左端と下側を回るため、搬送区間がもっとも長い（19.2 秒）。荷物 C は倉庫エリアに近く、搬送は 8.3 秒で済む。

## 3. 走行結果

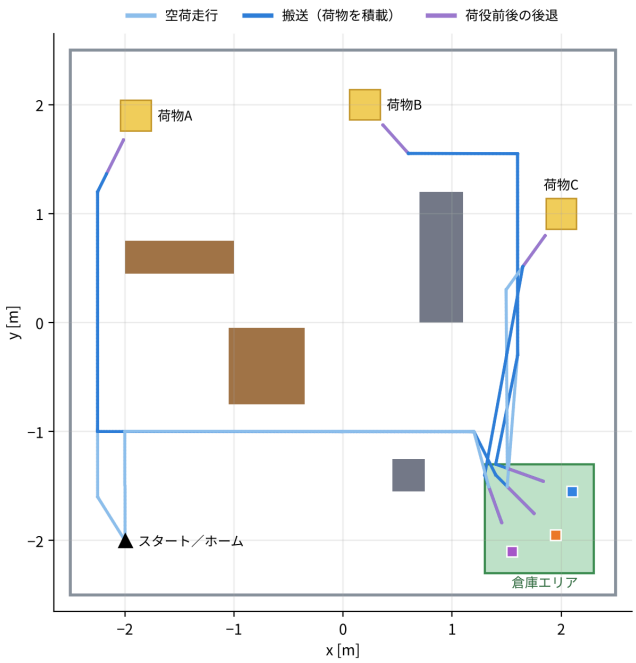



図1　走行軌跡（上から見た図）。小さい四角は荷降ろし後の荷物の位置。

### 3.1 軌跡

経路は、その場旋回と直進を組み合わせた折れ線になる。差動二輪は旋回中に並進しないため、各頂点で向きを変えてから直進する。

荷物は倉庫エリア内の別々の置き場所（スロット）に降ろす。目標位置との誤差は次のとおりで、いずれも約 1 mm に収まった。

| 荷物 | 目標との誤差 |
|---|---|
| A | 0.86 mm |
| B | 1.23 mm |
| C | 0.94 mm |
| ホームへの帰着 | 0.92 mm |

障害物と壁に対するロボット外形の最小クリアランスは **38.5 mm** で、接触はない。荷物を積み込むときだけは、荷物に寄せるのが目的なのでクリアランスがマイナスになる。

### 3.2 速度と指令

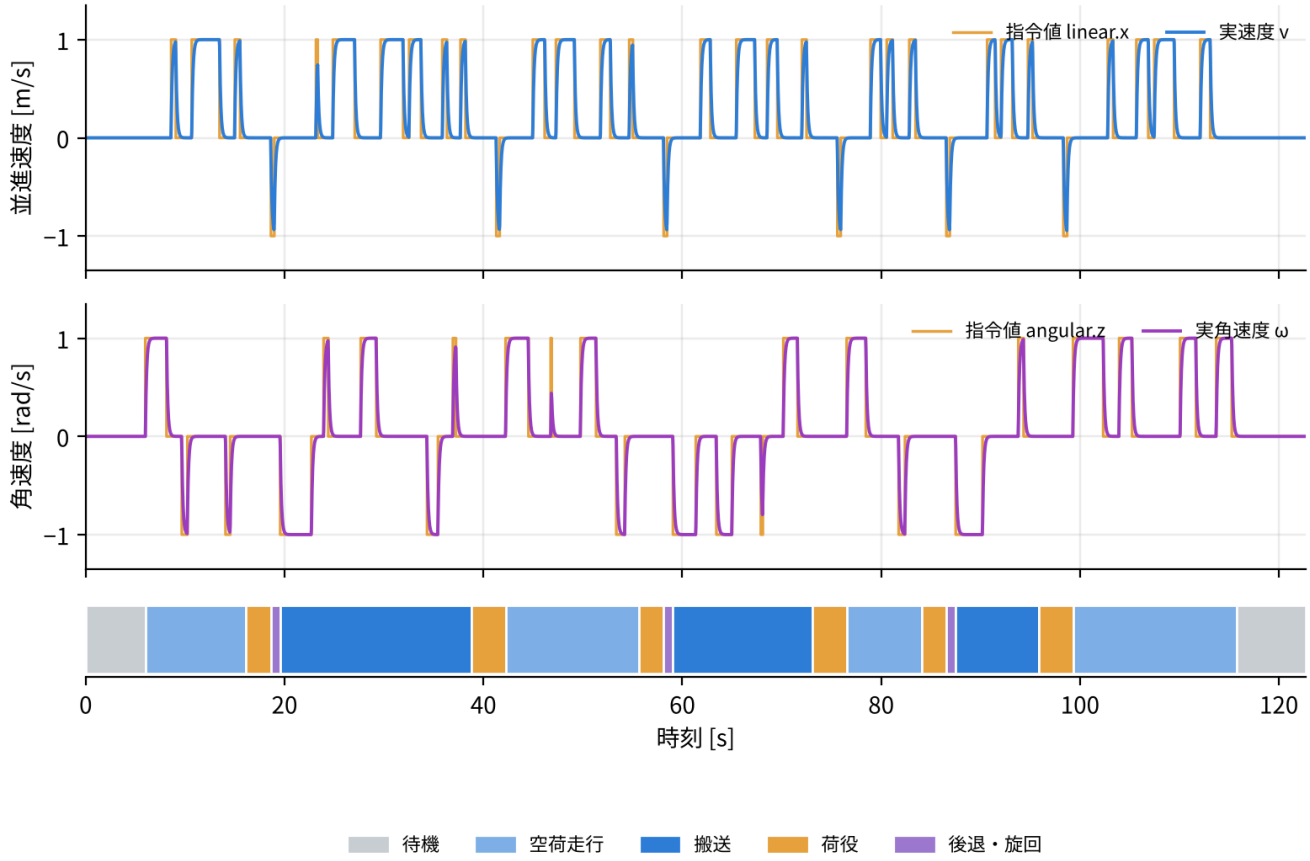



図2　並進速度・角速度の指令値（橙）と実際の値、および動作区間。指令は ±1 と 0 の三値で、実際の値は一次遅れで追従する。

キー指令は 114 回で、内訳はスペース（停止）57、W（前進）27、A（左旋回）14、D（右旋回）10、S（後退）6 である。最大並進速度は 1.0 m/s、最大車輪角速度は 30.3 rad/s（= 1.0 / 0.033）。後退（S）は積み込みと荷降ろしの直後だけに使い、荷物や置き場所から離れるために使っている。

## 4. コードの構成

### 4.1 処理の流れ

**① シミュレーション**
`Sim / simulate()`
キー指令の列から運動を積分し、30 fps ごとに状態を記録

→

**② 状態の保存**
`sim.pkl`
3,682 フレーム分の姿勢、速度、荷物、ログ

→

**③ 3D 描画**
`cameras() / render3d()`
カメラ補間、投影、影、ペインタ法

→

**④ 画面表示**
`hud() / card()`
パネル、ミニマップ、ログ、タイトル

→

**⑤ 書き出し**
`ffmpeg (libx264)`
2 プロセスで並列に描画して結合

シミュレーションと描画を分けているため、ルートを変えたときは ① だけを再実行して静止画で確認し（`still` モード）、問題がなければ全体を描画する、という流れで作業できる。

### 4.2 モジュール一覧

| 部分（行） | 主な関数・クラス | 役割 |
|---|---|---|
| 設定（1–31） | `R, SEP, TAU, KEYS, OBST …` | 記事のロボット仕様、キー割り当て、フィールド配置 |
| シミュレーション（33–186） | `Sim.run / press / goto / turn_to / approach / reverse / load / unload`、`simulate`、`check_collisions` | 運動学の積分、キー操作によるルートの記述、荷物の受け渡しアニメーション、衝突チェック |
| 形状（188–280） | `face, box, prism, robot_items, hull, shadow_of` | ポリゴンの生成（直方体と多角柱）、ロボットのモデル、影の輪郭（凸包） |
| カメラと描画（281–420） | `Cam, clip, poly3, line3, shade, render3d` | 透視投影、ニアクリップ、陰影、物体単位のペインタ法、2 倍解像度での描画 |
| 画面表示（422–540） | `hud, card, mm` | 情報パネル、キー表示、速度バー、ミニマップ、ログ、タイトルと完了表示 |
| 実行制御（542–598） | `cameras, render_range, __main__` | カメラの補間、ffmpeg へのフレーム出力、`sim / still / render` の 3 モード |

### 4.3 運動学と積分

差動二輪の運動は次の式で表される。$v$ は並進速度、$\omega$ は角速度、$r = 0.033$ m、$b = 0.19$ m である。

$$\dot{x} = v\cos\theta,\quad \dot{y} = v\sin\theta,\quad \dot{\theta} = \omega \qquad \omega_L = (v - \omega\, b/2)/r,\quad \omega_R = (v + \omega\, b/2)/r$$

指令値に対する応答は時定数 $\tau = 0.12$ s の一次遅れとし、1/480 s 刻みで積分する。一次遅れは指数関数で厳密に更新し、位置は中点の向きを使って更新することで、刻み幅による誤差を抑えている。

```
# Sim.run() より抜粋
k = 1 - math.exp(-h / TAU)          # 一次遅れの厳密な離散化
s.v += (s.vc - s.v) * k
s.w += (s.wc - s.w) * k
thm = s.th + s.w * h / 2            # 中点の向きで並進を積分
s.x += s.v * math.cos(thm) * h
s.y += s.v * math.sin(thm) * h
s.th = wrap(s.th + s.w * h)
```

### 4.4 キー操作による自動操縦

ルートは、teleop_node と同じキーを押す時間で記述する。指令は常に大きさ 1（1 m/s または 1 rad/s）なので、距離 $d$ [m] 進むには W を $d$ 秒、角度 $\phi$ [rad] 回るには A か D を $\phi$ 秒押してから停止すればよい。

```
def goto(s, px, py):
    s.turn_to(math.atan2(py - s.y, px - s.x))   # 目標の方向へその場旋回
    d = math.hypot(px - s.x, py - s.y)
    s.press('w'); s.run(d); s.press(' '); s.run(0.6)   # d 秒前進 → 停止 → 静定待ち
```

一次遅れがあっても、この開ループ制御で目標に正確に届く。一次遅れ系では出力の時間積分が入力の時間積分に等しく、立ち上がりで遅れた分を停止後の惰性がちょうど補うためである。

$$\int_0^\infty v(t)\,dt = \int_0^\infty v_c(t)\,dt = 1 \times d$$

第 3 章の約 1 mm の誤差は、旋回の不感帯（0.015 rad 未満は補正しない）と、停止後の静定待ちが有限であることから生じる残差である。

### 4.5 3D 描画

- **投影：**カメラ座標へ変換したあと、ピンホールモデルで透視投影する（画角 58°）。カメラより手前の面はニア面（0.05 m）で切り取る。
- **陰影：**平行光源と面の法線の内積で明るさを決め、背面は描かない。
- **影：**物体の頂点を光源の方向に沿って床へ投影し、その凸包を半透明で塗る。
- **描画順：**ペインタ法を物体単位で行う。まず物体を距離の順に並べ、次に物体内の面を並べる。面単位だけで並べると、床に置いた荷物がロボットより手前に描かれる不具合が出たため、この方式に改めた。積載中の荷物はロボットと同じ物体として扱う。
- **アンチエイリアス：**2560×1440 で描画してから 1/2 に縮小する。
- **カメラ：**追従カメラはロボットの向きに時定数 0.45 s で遅れて追従し、急な旋回でも画面が揺れにくい。冒頭と最後は俯瞰視点との間を滑らかに切り替える。

## 5. 検証

| 項目 | 方法 | 結果 |
|---|---|---|
| 衝突 | 全フレームでロボット外形上の 12 点と障害物・壁との距離を計算 | 最小 38.5 mm（接触なし） |
| 位置の精度 | 荷物の最終位置と置き場所、最終姿勢とホームの距離 | いずれも約 1 mm 以内 |
| 見た目 | 途中の代表フレームを静止画にして目視で確認 | 描画順の不具合とミニマップの文字のはみ出しを見つけて修正 |
| 動画ファイル | ffprobe で長さとフレーム数、結合後のフレームを確認 | 122.7 秒、3,682 フレーム、継ぎ目の欠落なし |

## 6. 制約と今後の課題

### 6.1 制約

- **Gazebo の物理演算ではない：**運動学モデルなので、車輪の滑り、慣性、接触力、キャスターの摩擦は再現していない。荷物の積み下ろしも、物理的な把持ではなく演出のためのアニメーションである。
- **開ループ制御：**理想的なモデルでは時間だけで正確に動けるが、実機や Gazebo では滑りや遅延で誤差が積み重なる。同じ手順を実機で使うには、オドメトリなどのフィードバックが要る。
- **描画の簡略化：**ペインタ法は、物体同士が入り組む配置では前後関係を誤る可能性がある。今回の配置では問題は出ていない。
- **処理時間：**描画は CPU だけで行うため、2 コアで約 6 分かかる。

### 6.2 今後の課題

- 同じ搬送ルートを ROS2 ノード（rclcpp）として実装し、記事の環境（Ubuntu 22.04、Humble、Fortress）の Gazebo 上で実行して録画する。

- Gazebo の `/odom` をブリッジしてフィードバック制御にし、本シミュレーションの結果（誤差約 1 mm）と比べる。
- 速度と加速度の制限を記事の値より現実的にし、搬送中の荷物の安定性を評価する。

## 付録　再現手順

```
sudo apt install ffmpeg fonts-noto-cjk fonts-dejavu-core
pip install numpy pillow
unzip robot_video_code.zip && cd robot_video_code
./build.sh                                    # → robot_cargo_task.mp4
python3 robot_task.py sim build/sim.pkl      # シミュレーションのみ（クリアランスも表示）
python3 robot_task.py still build/sim.pkl build 470 1150 3500   # 指定フレームを静止画に
```

参考：@N622「gazeboとROS2を連携してロボットを操作する」Qiita（2023-12-19）
https://qiita.com/N622/items/9d89f77d85d9da0af29e